Approximate Minimax Q Simulation Experiments: Codex Implementation Specification

0. Purpose of This Document

This document is a Codex-facing implementation specification for a publishable simulation experiment repository.

The repository should implement and compare three methods for two-player zero-sum adversarial queueing/control Markov games with countably infinite or large state spaces:

1. **NNQ**: neural-network minimax Q-learning; a strong and fair baseline.
2. **AMQ**: approximate minimax Q-learning with linear or polynomial feature-based function approximation.
3. **BVI**: bounded-state minimax value iteration / dynamic-programming-style solver.

The methods should be evaluated on three benchmarks:

1. **Routing system**
2. **Polling system**
3. **Service-rate-control system**

The final codebase must be structured as a reusable, reproducible research repository suitable for later publication-quality experiments and GitHub release.

---

1. High-Level Research Motivation

The original AMQ experiments were not broad or statistically rigorous enough. The new experimental project should address this by:

- Adding a third benchmark, service-rate control, that is related to but structurally different from routing and polling.
- Implementing a stronger NNQ baseline instead of relying on a weak two-layer ReLU network.
- Implementing BVI as a bounded-state dynamic-programming-style solver, while not hard-coding it as the ground truth.
- Comparing all three methods under the same environment interface, action semantics, cost definitions, evaluation protocol, random seeds, and plotting pipeline.
- Reporting robust metrics, including average defender cost, Bellman residual, policy consistency, convergence behavior, runtime, sample efficiency, and confidence intervals.

A key design principle:

> BVI should be treated as an approximate bounded-state solver, not as the automatically correct ground truth. NNQ and BVI should both be implemented carefully; later experiments determine which one is the stronger reference method.

---

2. Terminology

2.1 Algorithms

- **NNQ**: Neural-network minimax Q-learning. This is a baseline algorithm, but it must be implemented strongly and fairly.
- **AMQ**: Approximate minimax Q-learning with hand-designed feature maps and linear/polynomial function approximation.
- **BVI**: Bounded-state value iteration / minimax dynamic programming solver on a truncated finite state space.

2.2 Benchmarks

- **Routing system**: The defender controls whether routing decisions are defended. If an attack succeeds, arrivals are routed to the longest queue instead of the shortest queue.
- **Polling system**: The defender protects polling decisions. If an attack succeeds, the server polls the shortest queue instead of the longest queue.
- **Service-rate-control system**: The defender chooses a service-rate level. If an attack succeeds, the realized service rate is forced to the lowest service-rate level.

2.3 Baseline vs Reference

Do not assume a fixed universal ground truth. Use the following language in code comments, result files, and plots:

- NNQ is a **learned neural baseline**.
- BVI is a **bounded-state approximate solver**.
- AMQ is the **main feature-based reinforcement-learning method**.
- The stronger reference method for paper framing should be determined after comparing NNQ and BVI empirically.

---

3. Repository Requirements

The codebase should be modular and reproducible. Avoid monolithic notebooks.

Recommended structure:

```
minimax_queueing_experiments/
  README.md
  pyproject.toml or requirements.txt
  configs/
    routing/
    polling/
    service_rate_control/
  src/
    adversarial_queueing/
      __init__.py
      envs/
        base.py
```

```
        routing.py
        polling.py
        service_rate_control.py
      algorithms/
        nnq.py
        amq.py
        bvi.py
        minimax_solver.py
      features/
        routing_features.py
        polling_features.py
        service_rate_features.py
      evaluation/
        metrics.py
        rollout.py
        bellman_residual.py
      plotting/
        learning_curves.py
        policy_plots.py
        tables.py
      utils/
        seeding.py
        logging.py
        config.py
        checkpointing.py
  experiments/
    train.py
    evaluate.py
    sweep.py
    reproduce_main_results.py
  scripts/
    run_routing.sh
    run_polling.sh
    run_service_rate_control.sh
  tests/
    test_envs.py
    test_minimax_solver.py
    test_bvi.py
    test_features.py
  results/
    .gitkeep
```

The package name `adversarial_queueing` is intentionally neutral. It should not privilege AMQ, BVI, or NNQ in the repository structure.

The `results/` directory should be excluded from version control except for `.gitkeep` or small example outputs.

---

4. Common Game Interface

All benchmarks should implement a shared environment interface. The algorithms should not contain benchmark-specific transition logic.

4.1 Base Environment API

Create an abstract base class, for example `BaseAdversarialQueueEnv`, with the following methods:

```
class BaseAdversarialQueueEnv:
    def reset(self, seed: int | None = None):
    """Return initial state."""

    def step(self, attacker_action: int, defender_action: int):
    """
    Execute one simulation step.

    Returns:
      next_state,
      reward_or_cost,
      info
    """

    def transition_probabilities(self, state, attacker_action: int, defender_action: int):
    """
    Return a finite dictionary mapping next_state -> probability or rate.
    Required for BVI and Bellman residual calculations.
    """

    def cost(self, state, attacker_action: int, defender_action: int, next_state=None):
    """Return defender cost / attacker reward for the zero-sum game."""

    def attacker_actions(self, state):
    """Return available attacker actions."""

    def defender_actions(self, state):
    """Return available defender actions."""

    def encode_state(self, state):
    """Return numeric state representation for NNQ and feature maps."""
```

4.2 Sign Convention

Use a single consistent sign convention across all code:

- Define `cost` as the defender's one-step cost.
- The defender minimizes cost.
- The attacker maximizes the same cost.
- NNQ, AMQ, and BVI should all solve the same minimax problem:

```
Q(s,a,b) = c(s,a,b) + gamma * E[V(s')]
V(s) = min_defender max_attacker Q(s,a,b)
```

or equivalently:

```
V(s) = min_b max_a Q(s,a,b)
```

Be consistent about action order. Recommended convention:

```
a = attacker action
b = defender action
Q(s, a, b)
```

4.3 Continuous-Time Convention

The queueing benchmarks are continuous-time systems:

- Arrivals follow Poisson processes.
- Service times are exponentially distributed.
- Equivalently, arrivals and service completions are generated by competing exponential clocks.

Do not introduce an arbitrary "minimum time unit" as the primary model. The default implementation should use **uniformization** to convert the continuous-time Markov chain into a discrete-time Markov game with self-loops. This gives a clean one-step transition model for AMQ, NNQ, BVI, Bellman residuals, and policy evaluation.

For a state-action pair `(s, a, b)`, let the CTMC transition rates be:

```
q(s' | s, a, b),   s' != s
q_out(s, a, b) = sum_{s' != s} q(s' | s, a, b)
```

Choose a uniformization rate `R` satisfying:

```
R >= max_{s,a,b} q_out(s,a,b)
```

For truncated BVI, the maximum is taken over the truncated state space. For simulation-based AMQ/NNQ, choose a valid global upper bound from model parameters, for example arrival rates plus the maximum possible total service rate.

Then define discrete transition probabilities:

```
P_U(s' | s,a,b) = q(s' | s,a,b) / R,        s' != s
P_U(s  | s,a,b) = 1 - q_out(s,a,b) / R
```

This self-loop probability represents "no real event occurs during this uniformized step."

For discounted continuous-time cost with discount rate `beta > 0`, use the exact uniformized discrete-time conversion:

```
gamma = R / (R + beta)
one_step_cost = instantaneous_cost / (R + beta)
V(s) = min_b max_a [ one_step_cost(s,a,b) + gamma * E_{P_U}[V(s')] ]
```

If an experiment config specifies a discrete discount factor `gamma` directly, then the corresponding continuous-time discount rate is:

```
beta = R * (1 / gamma - 1)
```

The code should store both `R` and either `beta` or `gamma` in the saved config so that results are reproducible.

Event-driven simulation may be implemented as an optional validation mode. In event-driven mode:

```
Delta_t ~ Exponential(q_out(s,a,b))
s' is sampled with probability q(s'|s,a,b) / q_out(s,a,b)
continuation_discount = exp(-beta * Delta_t)
transition_cost = instantaneous_cost * (1 - exp(-beta * Delta_t)) / beta
```

However, the main experiments should default to uniformization to avoid ambiguity about time discretization.

---

5. Minimax Solver

Implement a shared minimax solver for one-state matrix games. This solver will be used by NNQ, AMQ, BVI, and evaluation code.

5.1 Required Function

```
def solve_zero_sum_matrix_game(payoff: np.ndarray, player: str = "defender") -> dict:
    """
    Solve a one-state zero-sum matrix game.

    Args:
      payoff: Array of shape [num_attacker_actions, num_defender_actions].
            Entries are defender costs / attacker rewards.
      player: Use "defender" to return the defender minimization value.

    Returns:
      {
```

```
        "value": float,
        "attacker_strategy": np.ndarray,
        "defender_strategy": np.ndarray,
    }
"""
```

5.2 Two-Action Closed Form

For binary attacker/defender actions, closed-form formulas may be used, but the implementation should still support generic finite actions through linear programming.

5.3 Linear Programming Fallback

Use `scipy.optimize.linprog` for general finite matrix games.

The solver must be tested on simple known games:

- Pure saddle point game.
- Matching-pennies-style mixed equilibrium.
- Dominant strategy cases.

---

6. Benchmark 1: Routing System

6.1 State

For `m` parallel queues:

```
x = (x_1, ..., x_m) ∈ \mathbb{Z}_{\ge 0}^m
```

where `x_i` is the number of jobs in queue/server `i`.

6.2 Actions

Attacker:

```
a ∈ {0, 1}
0 = not attack
1 = attack
```

Defender:

```
b ∈ {0, 1}
0 = not defend
1 = defend
```

6.3 Dynamics

- Jobs arrive according to a Poisson process with rate `lambda`.
- Service time at server `i` is exponentially distributed with service rate `mu_i`. Equivalently, service-completion events occur according to an exponential clock with rate `mu_i` when the queue is nonempty.
- If `a = 1` and `b = 0`, the attack succeeds and the arrival is routed to one of the longest queues.
- Otherwise, the arrival is routed to one of the shortest queues.
- Ties should be broken uniformly unless explicitly configured otherwise.

6.4 Cost

Default defender cost:

```
c(x,a,b) = ||x||_1 - c_attack * a + c_defend * b
```

The code should allow alternative congestion costs, e.g. quadratic cost, through configuration.

6.5 Default Configurations

Support at least:

- 3-server routing.
- 6-server routing if computationally feasible for AMQ and NNQ.
- Smaller bounded versions for BVI sensitivity tests.

---

7. Benchmark 2: Polling System

7.1 Modeling Source and Non-Negotiable Details

The nominal polling rule is not an invented modeling choice in this specification. It comes from the AMQ paper's polling experiment setup: in the absence of attacks, the server uses a longest-queue polling policy, meaning it selects the queue with the largest queue length. The same section states that if the polling decision is attacked and not defended, the server is redirected to the shortest queue instead.

Switching cost is a key part of this benchmark and must not be dropped. Every time the server changes from its current polled queue to a different queue, the implementation must charge a switching cost.

7.2 State

For n queues and one server, the state must include both queue lengths and the current server position:

$$ s = (x, p), x = (x_1, ..., x_n) ∈ ℤ_{⩾0}^n, p ∈ {0,1,...,n-1}. $$

Here x_i is the queue length of queue i, and p is the index of the currently polled queue.

Do not omit p in the implementation. Without p, the code cannot correctly compute whether a queue switch occurred, so it cannot correctly apply the switching cost.

7.3 Actions

Attacker:

- $a = 0$: not attack.
- $a = 1$: attack.

Defender:

- $b = 0$: not defend.
- $b = 1$: defend.

7.4 Nominal and Malicious Polling Decisions

Let $L(x)=arg max_i x_i$ be the set of longest queues, and let $S(x)=arg min_i x_i$ be the set of shortest queues.

The next intended polling target is:

- If $a = 1$ and $b = 0$, the attack succeeds and the malicious target is selected from $S(x)$.
- Otherwise, the nominal target is selected from $L(x)$.

Ties should be broken uniformly at random unless a config explicitly specifies a different tie-breaking rule.

Let p' denote the selected next polling target. The switch indicator is:

$$ n_{switch} = 1[p' ≠ p]. $$

The implementation should update the server position to p' after the polling decision.

7.5 Dynamics

Jobs arrive at queue i according to a Poisson process with rate $λ_i$.

Service time at the currently polled queue is exponentially distributed with service rate $μ$. Equivalently, service completion events occur according to an exponential clock with rate $μ$ when the selected queue is nonempty.

The service completion should apply to the selected polling target p' after the nominal or malicious polling decision is resolved. If $x_{p'} = 0$, service completion should not create a negative queue length.

The default implementation should use the uniformization convention from Section 4.3. For polling, a valid uniformization rate can be chosen as:

$$ R ⩾ Σ_i λ_i + μ. $$

7.6 Cost

The polling benchmark must include both queue-performance cost and switching cost.

A configurable default cost is:

$$ c(x,p,a,b,p') = c_{queue}(x) + C_{switch} 1[p' ≠ p] - c_{attack}a + c_{defend}b. $$

The original AMQ polling experiment used a load-distribution fairness-style term and switching cost. Support the following configurable queue-cost options:

- `sum`: $c_{queue}(x)=Σ_i x_i$.
- `sum_square`: $c_{queue}(x)=Σ_i x_i^2$.
- `fairness`: use the fairness-style term from the AMQ polling setup, with clear documentation of the sign convention.

The sign convention in this repository remains: this is defender cost / attacker reward. The defender minimizes it, and the attacker maximizes it.

---

8. Benchmark 3: Service-Rate-Control System

This is the new third benchmark. It is important because it is similar to queueing control but not another routing/scheduling task. The controlled object is service speed.

8.1 Initial Version

Start with a single-queue service-rate-control game.

State:

```
x ∈ \mathbb{Z}_{\ge 0}
```

where `x` is queue length.

8.2 Actions

Attacker:

```
a ∈ {0, 1}
0 = not attack
1 = attack
```

Defender:

```
b ∈ {0, 1, 2}
0 = low service rate
```

```
1 = medium service rate
2 = high service rate
```

The service rates are:

```
mu_levels = [mu_low, mu_medium, mu_high]
```

Initially use exactly three service-rate levels. Keep the code general enough to allow more levels later.

8.3 Effective Attack Rule

Attack succeeds when:

```
a = 1 and defender does not successfully defend / mitigate
```

For this benchmark, defender action is service-rate choice. The intended attack effect is:

> If the attack is effective, the realized service rate is forced to the lowest service rate level.

Therefore:

```
if a == 1 and attack_effective:
    realized_mu = mu_low
else:
    realized_mu = mu_levels[b]
```

Because the defender action here is not binary defend/not-defend but a service-rate level, define the effective attack logic carefully. Recommended initial interpretation:

```
if a == 1 and b < high_level:
    realized_mu = mu_low
else:
    realized_mu = mu_levels[b]
```

However, this interpretation should be configurable. Alternative interpretation:

```
if a == 1:
    realized_mu = mu_low
else:
    realized_mu = mu_levels[b]
```

If using the alternative, the defender can only compensate indirectly by choosing high service rate in non-attacked states; it cannot block an active attack. Prefer the first interpretation if we want service-rate selection to function as both control and defense intensity.

8.4 Recommended Final Interpretation for Initial Experiments

Use the following default because it preserves attacker-defender interaction:

```
b = 0: low service rate, no defense
b = 1: medium service rate, partial defense but attack still forces low service rate
b = 2: high service rate, robust defense; attack does not force service rate to low
```

Then:

```
if a == 1 and b in {0, 1}:
    realized_mu = mu_low
else:
    realized_mu = mu_levels[b]
```

This creates a meaningful trade-off:

- Low service rate saves operating cost but is vulnerable.
- Medium service rate improves service but remains vulnerable.
- High service rate is expensive but robust.

If the user later chooses a different formal rule, update only the environment config and documentation.

8.5 Dynamics

Jobs arrive according to a Poisson process with rate `lambda_arrival`. Service times are exponentially distributed. The active service rate is the realized rate after accounting for the defender's selected service level and any effective attack.

Use the continuous-time convention from Section 4.3. The default implementation should use uniformization, not an arbitrary fixed `dt`.

For the single-queue model, the non-self transition rates are:

```
q(x + 1 | x,a,b) = lambda_arrival
q(max(x - 1, 0) | x,a,b) = realized_mu(a,b)    if x > 0
```

When `x = 0`, service completion should not create a negative queue length. Either omit the service transition or treat it as a self-loop.

Choose:

```
R >= lambda_arrival + max(mu_levels)
```

Then the uniformized probabilities are:

```
P(x + 1 | x,a,b) = lambda_arrival / R
P(x - 1 | x,a,b) = realized_mu(a,b) / R        if x > 0
P(x     | x,a,b) = 1 - lambda_arrival/R - realized_mu(a,b)/R   if x > 0
P(x     | x,a,b) = 1 - lambda_arrival/R                         if x = 0
```

For BVI with truncation bound `B`, apply the configured boundary rule when an arrival at `x = B` would produce `B + 1`.

8.6 Cost

Use quadratic congestion cost by default.

Recommended default:

```
c(x,a,b) = q_congestion * x^2 + service_cost[b] - attack_cost * a
```

where:

```
service_cost[0] < service_cost[1] < service_cost[2]
```

The attacker receives this cost as reward; the defender minimizes it.

Important: tune parameters so that the optimal policy is not degenerate.

Avoid:

- Always choosing high service rate.
- Always choosing low service rate.
- Attack being so strong that every policy fails.

8.7 Expected Policy Shape

The service-rate-control benchmark should be used to test whether learned policies have threshold structure:

```
small queue length    -> low service rate
medium queue length   -> medium service rate
large queue length    -> high service rate
```

Plot:

```
x-axis: queue length
 y-axis: probability of choosing each service-rate level
```

Also compare policy behavior under different attack costs and attack intensities.

9. AMQ Algorithm Specification

9.1 Function Approximation

AMQ approximates Q by:

```
Q_w(s,a,b) = phi(s,a,b)^T w
```

where `phi` is a hand-designed feature vector.

9.2 Update Rule

The AMQ implementation should follow the algorithmic structure in the AMQ paper, not a generic simplified Q-learning update.

AMQ uses an off-policy temporal-difference update with a fixed behavior policy pair $(α,β)$ that samples actions with full support:

$$ A_k ∼ α(. | X_k), B_k ∼ β(. | X_k). $$

The approximate action-value function is linear in features:

$$ Q_w(s,a,b) = ϕ(s,a,b)^T w. $$

After sampling (X_k,A_k,B_k), the environment returns the one-step cost r_k and next state X_{k+1}. Under the uniformization convention, r_k should be the uniformized one-step cost, not an arbitrary fixed-dt cost.

The TD error is:

$$ Δ_k = r_k + γ V_w(X_{k+1}) - Q_{w_k}(X_k,A_k,B_k). $$

The next-state minimax value is computed from the matrix game induced by $Q_{w_k}(X_{k+1},a,b)$:

$$ V_w(X_{k+1}) = min_{σ ∈ Δ(B)} max_{a ∈ A} Σ_{b ∈ B} σ(b | X_{k+1}) Q_w(X_{k+1},a,b). $$

The defender mixed strategy $σ(. | X_{k+1})$ should be obtained by solving the linear program used in the AMQ paper's appendix:

$$ min_{σ,c} c $$

subject to:

$$ Σ_{b ∈ B} σ(b | X_{k+1})Q_w(X_{k+1},a,b) ⩽ c, for all a ∈ A, $$

$$ Σ_{b ∈ B} σ(b | X_{k+1}) = 1, $$

$$ σ(b | X_{k+1}) ⩾ 0, for all b ∈ B. $$

Then update:

$$ w_{k+1} = w_k + η_k ϕ(X_k,A_k,B_k)Δ_k. $$

Implementation requirements:

1. Keep the behavior policies $(α,β)$ separate from the target minimax policy solved inside the TD target.
2. The behavior policies must assign positive probability to all actions that should be explored.
3. The minimax backup must call the shared matrix-game solver or an equivalent LP routine.
4. The defender's target mixed strategy in the backup should be conditioned on the next state X_{k+1}, because the continuation value is evaluated at X_{k+1}.
5. Log TD error, weight norm, feature norm, and minimax value during training.
6. Support both pure minimax backup and mixed-strategy LP backup, but use the mixed-strategy LP backup as the default.

Pseudocode:

```
initialize weights w_0
for k = 0, 1, 2, ...:
    observe state X_k
    sample attacker action A_k ~ alpha(. | X_k)
    sample defender action B_k ~ beta(. | X_k)
    step environment and observe one-step cost r_k and next state X_{k+1}
    build payoff matrix M[a,b] = Q_w(X_{k+1}, a, b)
    solve defender minimization matrix game or LP to obtain V_w(X_{k+1})
    Delta_k = r_k + gamma * V_w(X_{k+1}) - Q_w(X_k, A_k, B_k)
    w_{k+1} = w_k + eta_k * phi(X_k, A_k, B_k) * Delta_k
```

9.3 Step Size

Support configurable schedules:

```
amq:
 learning_rate_schedule: "robbins_monro"
 eta0: 0.1
 decay_power: 0.6
```

Also allow constant learning rate for engineering tests.

9.4 Feature Maps

Feature maps are part of the experimental design and must be documented carefully. Do not present service-rate-control features as if they were copied from the original AMQ paper.

For routing and polling, the AMQ paper used two feature families:

- AMQ1: affine functions of the traffic state.
- AMQ2: second-order polynomial functions of the traffic state.

The new implementation should reproduce this idea as configurable feature sets, but it may clean up naming and implementation details.

For service-rate control, there is no existing feature map from the original AMQ paper. The feature set below is an initial engineering proposal based on three modeling facts:

1. The state is a single queue length x.
2. The cost is quadratic in congestion.
3. The defender action is a finite service-rate level, and the learned policy is expected to have a threshold-like dependence on x.

Therefore, implement service-rate-control features as configurable alternatives rather than one hard-coded basis.

Recommended initial service-rate-control feature sets:

Feature set A: basic quadratic state-action features

$$ ϕ(x,a,b) = [1, x, x^2, a, b, ab]^T. $$

This is small, easy to debug, and matches the quadratic congestion cost.

Feature set B: one-hot action interaction features

Use one-hot encodings e_a and e_b for attacker and defender actions, then include:

$$ 1, x, x^2, e_a, e_b, x e_b, x^2 e_b, e_a ⊗ e_b. $$

This gives different queue-length slopes and curvatures for different service-rate levels, which is useful if low, medium, and high service rates have different value-growth behavior.

Feature set C: ablation features

Also support ablations:

- state only: $[1,x,x^2]$ plus action indicators;
- linear only: remove x^2 terms;
- no action interaction: remove $x e_b$, $x^2 e_b$, and $e_a ⊗ e_b$.

The final paper should report which feature set is used and include an ablation or sensitivity check. If a feature set is hand-designed, say so explicitly.

Keep feature normalization configurable. Save feature-set name and normalization parameters in every experiment output.

9.5 Projection / Clipping

Support optional weight clipping or projection for numerical stability. If used, log it clearly.

---

10. NNQ Algorithm Specification

NNQ must be implemented as a strong and fair baseline.

10.1 Network Input

Input should include:

```
encoded_state
attacker_action encoding
defender_action encoding
```

Output can be either:

Option A:

```
Q(s,a,b) scalar for one state-action pair
```

Option B:

```
Q matrix for all (a,b) pairs at state s
```

Prefer Option B for small finite action spaces because it makes minimax backup efficient.

10.2 Network Architecture

Make architecture configurable:

```
nnq:
  hidden_sizes: [128, 128, 128]
  activation: "relu"
  layer_norm: false
  residual: false
```

Support sweeps over:

- Hidden sizes.
- Number of layers.
- Learning rate.
- Target update frequency.

- Batch size.
- Replay buffer size.

10.3 Training Components

Required:

- Replay buffer.
- Target network.
- Adam or AdamW optimizer.
- Gradient clipping.
- Mini-batch updates.
- Epsilon-greedy or exploratory behavior policy with full action support.
- Periodic evaluation rollouts.

10.4 Target

For transition `(s,a,b,c,s')`:

```
y = c + gamma * min_b' max_a' Q_target(s', a', b')
loss = mse(Q_online(s,a,b), y)
```

For mixed strategies, use the matrix-game value rather than pure `min max` if randomized equilibrium is needed.

---

11. BVI Algorithm Specification

11.1 Core Principle

BVI solves a finite truncated game:

```
S_B = {states satisfying queue lengths <= B}
```

BVI should not be described as exactly solving the original unbounded game.

Use names like:

```
V_B
Q_B
policy_B
```

not:

```
V_star_exact
Q_star_exact
true_policy
```

11.2 State Truncation

For single-queue service-rate control:

```
S_B = {0, 1, ..., B}
```

For multi-queue systems:

```
S_B = {0, 1, ..., B}^m
```

Be careful: multi-queue BVI scales exponentially and may only be feasible for small `m` and small `B`.

11.3 Boundary Handling

Implement configurable boundary modes:

Mode 1: clip

```
next_x = min(next_x, B)
```

Simple but underestimates high-congestion risk.

Mode 2: clip_with_penalty

```
next_x = min(next_x, B)
cost += overflow_penalty
```

More conservative but introduces a penalty parameter.

Mode 3: reject_or_absorb

Optional. Can be implemented later.

Default:

```
bvi:
  boundary_mode: "clip"
```

11.4 Evaluation Interior

Because boundary states are distorted by truncation, evaluate BVI primarily on interior states:

```
bvi:
  max_queue_length: 200
  eval_max_queue_length: 150
```

Do not overinterpret states close to `B`.

11.5 Sensitivity Checks

Run BVI for multiple values of `B`:

```
B ∈ {50, 100, 150, 200}
```

Check whether:

- Value estimates stabilize.
- Policy thresholds stabilize.
- Bellman residual decreases.
- Boundary hit rate is small during evaluation.

Only if these checks pass should BVI be treated as a strong approximate reference.

11.6 Value Iteration Update

For each state `s`:

```
Q_B(s,a,b) = c(s,a,b) + gamma * sum_{s'} P_B(s'|s,a,b) V_B(s')
V_B(s) = value_of_matrix_game(Q_B(s,:,:))
```

Stop when:

```
max_s |V_new(s) - V_old(s)| < tolerance
```

Log number of iterations and final residual.

---

12. Evaluation Metrics

All algorithms must be evaluated with the same protocol.

12.1 Primary Metrics

1. **Average defender cost**

2. Estimated through rollouts.

3. Report mean and 95% confidence interval over seeds.

4. **Discounted cumulative cost**

5. Useful for comparison with previous experiments.

6. **Bellman residual**

7. Independent of NNQ.

8. Important because previous experiments over-relied on policy consistency with NNQ.

9. **Policy consistency**

10. Compare policies pairwise: AMQ vs NNQ, AMQ vs BVI, NNQ vs BVI.

11. Do not only compare to NNQ.

12. **Runtime**

13. Wall-clock training time.

14. Value iteration runtime.

15. **Sample efficiency**

16. Number of transitions needed to reach a given performance or residual threshold.

17. **Convergence curves**

18. Training loss.

19. TD error.
20. Bellman residual.
21. Average evaluation cost.

12.2 Service-Rate-Control-Specific Metrics

- Learned threshold locations.
- Probability of low / medium / high service rate as a function of queue length.
- Boundary hit rate under learned policy.
- Stability proxy: long-run average queue length.
- Tail metric: probability that queue length exceeds a threshold.

12.3 Statistical Reporting

For each major result:

```
number_of_seeds >= 10
```

Report:

- Mean.
- Standard deviation.
- 95% confidence interval.

Optional but recommended:

- Wilcoxon signed-rank test or paired bootstrap comparison for key method comparisons.

---

13. Plot Requirements

Generate publication-quality figures automatically from saved result files.

13.1 Common Plots

- Learning curves with confidence bands.
- Bellman residual vs training steps.
- Average cost vs training steps.
- Runtime comparison.
- Policy consistency matrix.

13.2 Routing / Polling Plots

- Policy heatmaps for two-dimensional projections.
- Normalized cost tables.
- Convergence comparison of NNQ, AMQ, and BVI where feasible.

13.3 Service-Rate-Control Plots

- Queue length vs service-level probabilities.
- Queue length vs attacker action probability.
- Threshold comparison across methods.
- Threshold shift under different attack costs or service costs.
- BVI sensitivity to `B`.

---

14. Config System

All experiments should be driven by YAML config files.

Example:

```yaml
env:
  name: "service_rate_control"
  gamma: 0.95
```

```yaml
  lambda_arrival: 2.0
  mu_levels: [1.0, 3.0, 5.0]
  service_costs: [0.0, 0.5, 2.0]
  attack_cost: 0.5
  congestion_cost: "quadratic"
  q_congestion: 1.0
  transition_model: "uniformized"
  uniformization_rate: 8.0

algorithm:
  name: "amq"
  feature_set: "quadratic_action_interaction"
  eta0: 0.1
  decay_power: 0.6
  total_steps: 100000

seeds: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

logging:
  eval_interval: 1000
  save_checkpoints: true
  output_dir: "results/service_rate_control/amq_default"
```

---

15. Reproducibility Requirements

The code must provide:

- Deterministic seeding for NumPy, PyTorch, and Python random.
- Saved config copy in every result directory.
- Saved metrics as CSV or JSONL.
- Saved final policy artifacts.
- Saved model checkpoints for NNQ and AMQ.
- Version metadata if possible.

Every experiment run should produce:

```
results/<benchmark>/<algorithm>/<timestamp_or_run_name>/
  config.yaml
  metrics.csv
  summary.json
  checkpoints/
  plots/
```

---

16. Testing Requirements

Write tests before large experiments.

16.1 Environment Tests

- State never becomes invalid.
- Transition probabilities sum to 1 for discrete-time/uniformized environments.
- Transition rates are nonnegative for CTMC environments.
- Costs are finite.
- All actions are available where expected.

16.2 Minimax Solver Tests

- Correct pure equilibrium.
- Correct mixed equilibrium.
- Strategy probabilities sum to 1.
- Returned value matches direct matrix-game conditions.

16.3 BVI Tests

- Value iteration converges on a tiny toy environment.
- Boundary handling works.
- Increasing `B` does not crash.

16.4 Algorithm Smoke Tests

- AMQ runs for a small number of steps.
- NNQ performs one training update.
- Evaluation loop runs for each benchmark.

---

17. Implementation Phases

Phase 1: Core Infrastructure

- Base environment interface.
- Minimax matrix-game solver.
- Config loader.
- Seeding utilities.
- Logging utilities.

Phase 2: Service-Rate-Control Benchmark

Implement this first because it is the cleanest BVI test case.

- Single-queue environment.
- AMQ features.
- BVI solver.
- NNQ baseline.
- Policy threshold plots.

Phase 3: Routing Benchmark

- Multi-queue routing environment.
- AMQ routing features.

- NNQ training.
- BVI only for small truncated cases.

Phase 4: Polling Benchmark

- Polling environment.
- AMQ polling features.
- NNQ training.
- BVI only for small truncated cases.

Phase 5: Sweeps and Publication Figures

- Hyperparameter sweeps for NNQ.
- BVI truncation sensitivity.
- Multi-seed runs.
- Final tables and plots.

---

18. Important Design Warnings

1. Do not hard-code BVI as ground truth.
2. Do not compare AMQ only to NNQ policy consistency.
3. Do not use a weak NNQ baseline.
4. Do not let BVI boundary artifacts dominate conclusions.
5. Do not mix benchmark transition logic into algorithm classes.
6. Do not rely on notebooks as the primary experimental pipeline.
7. Do not report single-seed results as final evidence.
8. Do not allow degenerate service-rate-control parameters where the best policy is always high or always low.

---

19. Open Modeling Decisions / TODOs

These should be resolved before final large-scale experiments.

19.1 Service-Rate-Control Attack Semantics

Current default proposal:

```
If attacker attacks and defender does not choose the robust/high service level,
then realized service rate is forced to mu_low.
```

Need to confirm whether:

- high service rate should be interpreted as defense,
- or there should be a separate binary defend action in addition to service-rate choice.

19.2 Continuous-Time vs Discrete-Time Simulation

Need to decide whether all environments should use:

- event-driven CTMC simulation,
- uniformized discrete-time transition model,
- or simple fixed-dt approximation.

Recommendation: use uniformization for BVI compatibility and consistent Bellman updates.

19.3 Routing and Polling BVI Scope

BVI may be infeasible for large 6-server systems due to exponential state growth. Need to define small-state BVI experiments separately from large-state AMQ/NNQ experiments.

19.4 Final Parameter Sets

Need final default parameters for:

- arrival rates,
- service rates,
- attack costs,
- defense/service costs,
- discount factor,
- truncation bounds,
- evaluation horizon.

---

20. Expected Final Experimental Claims

The code should make it possible to test, but not assume, the following claims:

1. AMQ learns competitive minimax policies with much fewer samples than NNQ.
2. A strong NNQ baseline may outperform weak NNQ from earlier experiments, so comparisons must be fair.
3. BVI is useful as a bounded-state reference when truncation sensitivity is controlled.
4. Service-rate control produces interpretable threshold policies.
5. Under attack risk, the defender's service-rate policy should become more conservative, switching to higher service rates earlier.
6. Across routing, polling, and service-rate control, AMQ should demonstrate robustness beyond one specialized routing-style task.